

Tasks on Auto Scaling Groups (ASG)

1. Create a VPC in the N. Virginia region.
2. Create two subnets, a public subnet and a private subnet.
3. Provide the IGW to the VPC.
4. Create a Public RT and a Private RT.
5. Deploy a NAT gateway on the public subnet and attach the NAT gateway to the private subnet.
6. Create two instances, one in the public subnet and one in the private subnet.

Step to create a VPC with Public/Private Subnets and NAT Gateway in AWS (us-east-1):

Objective

Create a **secure and scalable VPC** with:

- A **public subnet** (for internet-facing resources).
- A **private subnet** (for backend resources).
- An **Internet Gateway (IGW)** for public subnet access.
- A **NAT Gateway** for private subnet outbound internet access.
- **Route Tables (RT)** to control traffic flow.
- **EC2 instances** in both subnets for testing connectivity.

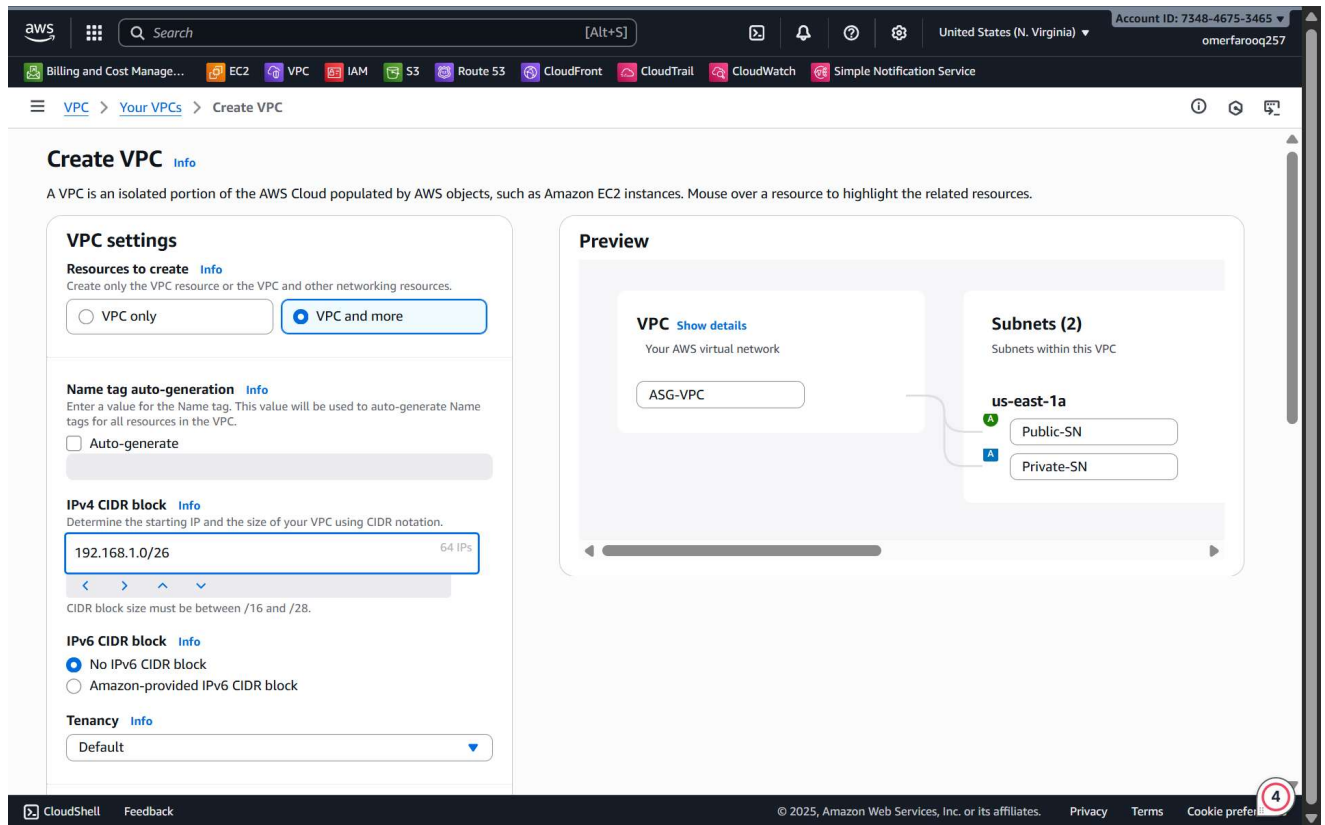
Step 1: Navigate to VPC Dashboard

1. Log in to the **AWS Management Console**.
2. Go to **Services** → **VPC** (under *Networking & Content Delivery*).
3. Click "**VPC and more**" under the "**Create VPC**" dropdown.

Step 2: Configure VPC Settings

1. VPC Name and CIDR Block

- **Name tag:** ASG-VPC
- **IPv4 CIDR block:** 192.168.1.0/26
- **IPv6 CIDR block:** *No IPv6 CIDR block* (optional).



2. Subnet Settings

- **Number of Availability Zones (AZs):** 2 (e.g., us-east-1a and us-east-1b).
- **Public Subnets:**
 - **CIDR blocks:** 192.168.1.0/27 (AZ1)
 - **Enable auto-assign public IPv4:** ☒ (Checked).
- **Private Subnets:**
 - **CIDR blocks:** 192.168.1.32/27 (AZ1)
 - **Enable auto-assign public IPv4:** ☐ (Unchecked).

3. NAT Gateway

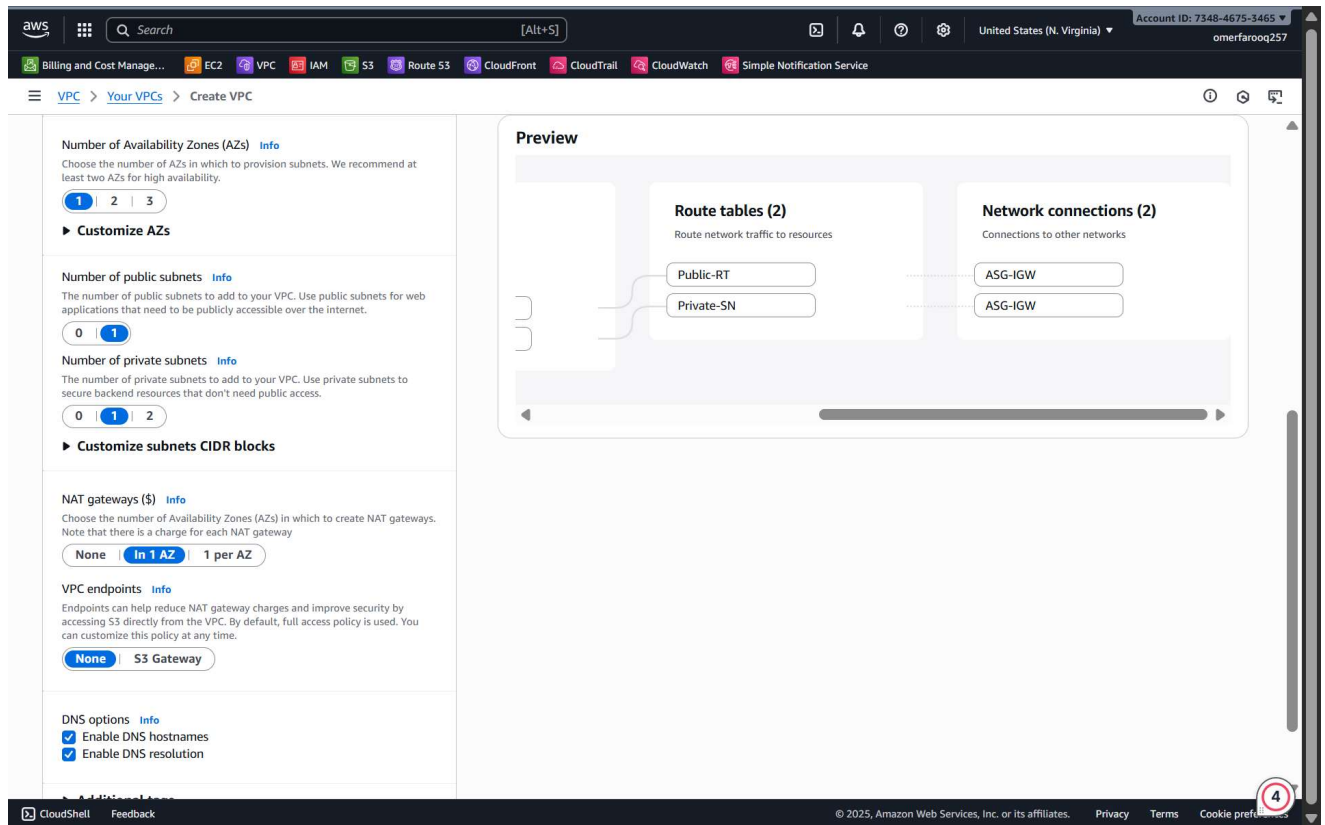
- **NAT Gateway configuration:** ☒ (Enable).
- **Elastic IP Allocation ID:** *Auto-assign* (or select an existing EIP).

4. VPC Endpoints (Optional)

- **S3 Gateway Endpoint:** ☒ (Enable for private subnet access to S3 without NAT).

5. DNS Options

- **Enable DNS hostnames:** ☒ (Checked).
- **Enable DNS resolution:** ☒ (Checked).



Step 3: Review and Create

1. Click "Create VPC".
2. AWS will automatically:
 - Create the **VPC**.
 - Create **public/private subnets** in 1 AZ.
 - Attach an **Internet Gateway (IGW)** to the VPC.
 - Create **public/private route tables** with appropriate routes.
 - Deploy a **NAT Gateway** in the public subnet.
 - Associate route tables with subnets.
3. Wait for the **VPC status** to show "**Available**" (takes ~2-5 minutes).

Create VPC workflow

⌂ Wait for NAT Gateways to activate

72%

▼ Details

- ✔ Create VPC: [vpc-08505c7558333a4cd](#)
- ✔ Enable DNS hostnames
- ✔ Enable DNS resolution
- ✔ Verifying VPC creation: [vpc-08505c7558333a4cd](#)
- ✔ Create subnet: [subnet-0c3d581c26e33c5cf](#)
- ✔ Create subnet: [subnet-0bdd33bdd78339269](#)
- ✔ Create internet gateway: [igw-061c4a3c63779ef30](#)
- ✔ Attach internet gateway to the VPC
- ✔ Create route table: [rtb-0f8964ff739dd983c](#)
- ✔ Create route
- ✔ Associate route table
- ✔ Allocate elastic IP: [eipalloc-08ebe4787c0ed0795](#)
- ✔ Create NAT gateway: [nat-0c9079c7fbf8bf607](#)
- ⌚ Wait for NAT Gateways to activate
- ⌚ Create route table
- ⌚ Create route
- ⌚ Associate route table
- ⌚ Verifying route table creation

Create VPC workflow

✔ Success

▼ Details

- ✔ Create VPC: [vpc-08505c7558333a4cd](#)
- ✔ Enable DNS hostnames
- ✔ Enable DNS resolution
- ✔ Verifying VPC creation: [vpc-08505c7558333a4cd](#)
- ✔ Create subnet: [subnet-0c3d581c26e33c5cf](#)
- ✔ Create subnet: [subnet-0bdd33bdd78339269](#)
- ✔ Create internet gateway: [igw-061c4a3c63779ef30](#)
- ✔ Attach internet gateway to the VPC
- ✔ Create route table: [rtb-0f8964ff739dd983c](#)
- ✔ Create route
- ✔ Associate route table
- ✔ Allocate elastic IP: [eipalloc-08ebe4787c0ed0795](#)
- ✔ Create NAT gateway: [nat-0c9079c7fbf8bf607](#)
- ✔ Wait for NAT Gateways to activate
- ✔ Create route table: [rtb-087373e644a6fa2a4](#)
- ✔ Create route
- ✔ Associate route table
- ✔ Verifying route table creation

View VPC

Step 4: Verify Resources

1. VPC:

- Go to **VPC Dashboard** → **Your VPCs** → Check ASG-VPC.

2. Subnets:

- Go to **Subnets** → Verify:
 - 2 **public subnets** (with auto-assign public IP enabled).
 - 2 **private subnets** (no public IP).

3. Internet Gateway:

- Go to **Internet Gateways** → Verify ASG-IGW is attached.

4. NAT Gateway:

- Go to **NAT Gateways** → Verify NAT Gateway is in the **public subnet**.

5. Route Tables:

- Go to **Route Tables** → Verify:
 - **Public RT:** Route to 0.0.0.0/0 → IGW.
 - **Private RT:** Route to 0.0.0.0/0 → NAT Gateway.

vpc-08505c7558333a4cd / ASG-VPC Actions

Details Info

VPC ID
vpc-08505c7558333a4cd

DNS resolution
Enabled

Main network ACL
acl-039163ea2fbd4bae0

IPv6 CIDR (Network border group)
-

State
Available

Tenancy
default

Default VPC
No

Network Address Usage metrics
Disabled

Block Public Access
Off

DHCP option set
dopt-09a84d39e5d39ac97

IPv4 CIDR
192.168.1.0/26

Route 53 Resolver DNS Firewall rule groups
-

DNS hostnames
Enabled

Main route table
rtb-052f6b5abdbab74f0

IPv6 pool
-

Owner ID
734846753465

[Resource map](#) | [CIDRs](#) | [Flow logs](#) | [Tags](#) | [Integrations](#)

Resource map Info Show all details

VPC
Your AWS virtual network
ASG-VPC

Subnets (2)
Subnets within this VPC
us-east-1a
Public-SN
Private-SN

Route tables (3)
Route network traffic to resources
Public-RT
rtb-052f6b5abdbab74f0
Private-SN

Network Connections (2)
Connections to other networks
ASG-IGW
ASG-IGW

Step 5: Launch EC2 Instances (Optional)

Public Instance:

1. Go to **EC2 Dashboard** → **Launch Instance**.
2. Configure:
 - **Name:** Public-Instance.
 - **AMI:** Amazon Linux 2 (ami-0c55b159cbfafa1f0).
 - **Instance Type:** t2.micro.
 - **Key Pair:** Select or create a new key pair.
 - **Network Settings:**
 - **VPC:** ASG-VPC
 - **Subnet:** Select a **public subnet** (e.g., 192.168.1.0/26).
 - **Auto-assign Public IP:** ☒ (Enabled).
 - **Security Group:** Allow **SSH (port 22)** and **HTTP (port 80)** from your IP.
3. Click **Launch Instance**.

Private Instance:

1. Repeat the steps for the **private instance**:

- **Name:** Private-Instance.
- **Subnet:** Select a **private subnet** (e.g., 192.168.1.32/27).
- **Auto-assign Public IP:** **✗** (Disabled).
- **Security Group:** Allow **SSH (port 22)** from the **public subnet CIDR** (192.168.1.0/27).

Public Instance:

i-0d627472082e88842 (Public Server)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

▼ Instance summary Info

Instance ID i-0d627472082e88842	Public IPv4 address 98.81.249.201 open address	Private IPv4 addresses 192.168.1.4
IPv6 address -	Instance state Running	Public DNS ec2-98-81-249-201.compute-1.amazonaws.com open address
Hostname type IP name: ip-192-168-1-4.ec2.internal	Private IP DNS name (IPv4 only) ip-192-168-1-4.ec2.internal	

Private Instance:

Instances (1/2) Info

Last updated 1 minute ago

Connect | Instance state | Actions | Launch instances

Find Instance by attribute or tag (case-sensitive) | All states

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public
☒	Private Server	i-0a1ede57ed707b746	Running	t2.micro	Initializing	View alarms +	us-east-1a	-
☐	Public Server	i-0d627472082e88842	Running	t2.micro	Initializing	View alarms +	us-east-1a	ec2-98-

i-0a1ede57ed707b746 (Private Server)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

▼ Instance summary Info

Instance ID i-0a1ede57ed707b746	Public IPv4 address -	Private IPv4 addresses 192.168.1.45
IPv6 address -	Instance state Running	Public DNS -

7. Deploy the Apache server on both the EC2 instances with the sample index.html file.

Steps to deploy the Apache server on both the EC2 Instances:

Objective:

- Install and configure **Apache (httpd)** on both the **public and private EC2 instances**.
- Create a **sample index.html** file to verify the web server is running.
- Test access to the Apache server from:
 - The **public instance** (directly via public IP).
 - The **private instance** (via SSH from the public instance).

Step 1: Install Apache on the Public Instance

1. Update the package manager:

```
# sudo yum update -y
```

2. Install Apache (httpd):

```
# sudo yum install httpd -y
```

3. Start the Apache service:

```
# sudo systemctl start httpd
```

4. Enable Apache to start on boot:

```
# sudo systemctl enable httpd
```

5. Verify Apache is running:

```
# sudo systemctl status httpd
```

- Expected output: active (running).

Step 3: Create a Sample index.html File (Public Instance)

1. Navigate to the Apache web directory and create a sample index.html file:

```
# vi /var/www/html/index
```

```

[root@ip-192-168-1-4 ec2-user]# yum install httpd
Last metadata expiration check: 0:01:54 ago on Tue Oct 28 13:28:54 2025.
Package httpd-2.4.65-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-192-168-1-4 ec2-user]# systemctl start httpd
[root@ip-192-168-1-4 ec2-user]# systemctl status httpd
bash: systemctl: command not found
[root@ip-192-168-1-4 ec2-user]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: active (running) since Tue 2025-10-28 13:31:00 UTC; 14s ago
     Docs: man:httpd.service(8)
  Main PID: 29403 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 177 (limit: 1106)
    Memory: 12.9M
       CPU: 63ms
    CGroup: /system.slice/httpd.service
            └─29403 /usr/sbin/httpd -DFOREGROUND
              └─29420 /usr/sbin/httpd -DFOREGROUND
                └─29421 /usr/sbin/httpd -DFOREGROUND
                  └─29422 /usr/sbin/httpd -DFOREGROUND
                    └─29423 /usr/sbin/httpd -DFOREGROUND

oct 28 13:31:00 ip-192-168-1-4.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
oct 28 13:31:00 ip-192-168-1-4.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
oct 28 13:31:00 ip-192-168-1-4.ec2.internal httpd[29403]: server configured, listening on: port 80
[root@ip-192-168-1-4 ec2-user]#

```

```

[root@ip-192-168-1-4 ec2-user]# vi /var/www/html/index.html
[root@ip-192-168-1-4 ec2-user]# cat /var/www/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Welcome Page</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
      background-color: #f4f4f9;
      color: #333;
    }
    h1 {
      font-size: 2.5em;
      font-weight: bold;
      color: #0066cc;
    }
    p {
      font-size: 2em;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1>Gulab, Khwab, Dawa, Zaher, Jaam, Kya Kya Hai?</h1>
  <h1>Main Aa Gaya Hon Bata Intezam Kya Kya Hai</h1>
</body>
</html>
[root@ip-192-168-1-4 ec2-user]#

```

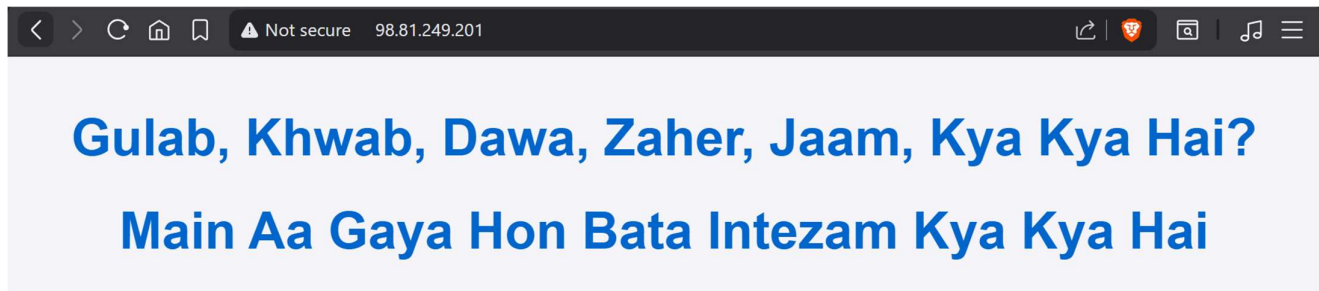
4. Restart Apache to apply changes:

```
# systemctl restart httpd
```

Step 4: Test Apache on the Public Instance

1. Open a web browser and navigate to:

https:98.81.249.201:80



Step 5: Connect to the Private Instance from the Public Instance

1. From the **public instance terminal**, SSH into the **private instance**:

ssh -i <KeyPair.pem> ec2-user@<PrivateInstancePrivateIP>

Step 6: Install Apache on the Private Instance

Repeat **Step 2** on the private instance:

sudo yum update -y

sudo yum install httpd -y

sudo systemctl start httpd

sudo systemctl enable httpd

sudo systemctl status httpd

Step 7: Create a Sample index.html File (Private Instance)

1. Navigate to the Apache web directory and create a sample index.html file:

cd /var/www/html/index.html

2. Restart Apache:

systemctl restart httpd

Step 8: Test Apache on the Private Instance

- **From the public instance**, test access to the private instance's Apache server:

curl http://<PrivateInstancePrivateIP>

```
[root@ip-192-168-1-45 ec2-user]# systemctl start httpd
[root@ip-192-168-1-45 ec2-user]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: active (running) since Tue 2025-10-28 13:11:37 UTC; 5s ago
     Docs: man:httpd.service(8)
  Main PID: 25920 (httpd)
    Status: "Started, listening on: port 80"
     Tasks: 177 (limit: 1106)
    Memory: 12.9M
       CPU: 59ms
    CGroup: /system.slice/httpd.service
            └─25920 /usr/sbin/httpd -DFOREGROUND
              └─25995 /usr/sbin/httpd -DFOREGROUND
                └─26001 /usr/sbin/httpd -DFOREGROUND
                  └─26006 /usr/sbin/httpd -DFOREGROUND
                    └─26007 /usr/sbin/httpd -DFOREGROUND

Oct 28 13:11:37 ip-192-168-1-45.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Oct 28 13:11:37 ip-192-168-1-45.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Oct 28 13:11:37 ip-192-168-1-45.ec2.internal httpd[25920]: Server configured, listening on: port 80
[root@ip-192-168-1-45 ec2-user]# |
```

```
[root@ip-192-168-1-45 ec2-user]# vi /var/www/html/index.html
[root@ip-192-168-1-45 ec2-user]# cat index.html
cat: index.html: No such file or directory
[root@ip-192-168-1-45 ec2-user]# cat /var/www/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Welcome Page</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
      background-color: #f4f4f9;
      color: #333;
    }
    h1 {
      font-size: 2.5em;
      font-weight: bold;
      color: #0066cc;
    }
    p {
      font-size: 2em;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1>Gulab, Khwab, Dawa, Zaher, Jaam, Kya Kya Hai?</h1>
  <h1>Main Aa Gaya Hon Bata Intezam Kya Kya Hai</h1>
</body>
</html>
[root@ip-192-168-1-45 ec2-user]#
```

```
[root@ip-192-168-1-4 ec2-user]# curl http://192.168.1.45
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Welcome Page</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
      background-color: #f4f4f9;
      color: #333;
    }
    h1 {
      font-size: 2.5em;
      font-weight: bold;
      color: #0066cc;
    }
    p {
      font-size: 2em;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1>Gulab, Khwab, Dawa, Zaher, Jaam, Kya Kya Hai?</h1>
  <h1>Main Aa Gaya Hon Bata Intezam Kya Kya Hai</h1>
</body>
</html>
[root@ip-192-168-1-4 ec2-user]# |
```

8. Create one application load balancer and attach the load balancer to both the EC2 instances.

Objective:

- Create an **Application Load Balancer (ALB)** to distribute traffic between the **public and private EC2 instances**.
- Ensure the ALB routes traffic to the **Apache servers** running on both instances.

Steps to launch an ALB and attach both (Public and Private) instances:

Step 1: Create a Security Group for the ALB

AWS Console:

1. Go to **EC2 Dashboard** → **Security Groups** → **Create Security Group**.
2. Configure:
 - **Security group name:** ALB-SG
 - **Description:** Security group for Application Load Balancer
 - **VPC:** ASG-VPC
3. **Add Inbound Rules:**
 - **Type:** HTTP (Port 80) **Source:** 0.0.0.0/0 (or restrict to your IP for security).
 - **Type:** HTTPS (Port 443) (*Optional*) **Source:** 0.0.0.0/0
4. Click **Create Security Group**.

Step 2: Create a Target Group for EC2 Instances

AWS Console:

1. Go to **EC2 Dashboard** → **Target Groups** → **Create Target Group**.
2. Configure:
 - **Target group name:** ASG-Tasks
 - **Target type:** Instance
 - **Protocol:** HTTP
 - **Port:** 80
 - **VPC:** ASG-VPC
3. **Register Targets:**
 - Select both **public and private EC2 instances**.
 - Click **Include as pending below** → **Create Target Group**.

EC2 > Target groups

Network & Security

Load Balancing

Target groups Info

Filter target groups

Name

ARN

Port

Protocol

Target type

No target groups

You don't have any target groups in us-east-1

Create target group

0 target groups selected

Select a target group above.

Step 1

Create target group

Step 2

Register targets

Create target group

Your load balancer routes requests to the targets in a target group and performs health checks on the targets.

Basic configuration

Settings in this section can't be changed after the target group is created.

Choose a target type

Instances

IP addresses

Lambda function

Application Load Balancer

Target group name

ASG-Task

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

EC2 > Target groups > Create target group

Target group name

ASG-Task

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Protocol

Protocol for load balancer-to-target communication. Can't be modified after creation.

HTTP

Port

Port number where targets receive traffic. Can be overridden for individual targets during registration.

80

1-65535

IP address type

Only targets with the indicated IP address type can be registered to this target group.

IPv4

IPv6

VPC

Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

vpc-08505c7558333a4cd (ASG-VPC)

192.168.1.0/26

Protocol version

HTTP1

Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

Create VPC

EC2 > Target groups > Create target group

Step 1
Create target group

Step 2
Register targets

Register targets

This is an optional step to create a target group. However, to ensure that your load balancer routes traffic to this target group you must register your targets.

Available instances (2/2)

Filter instances

☒	Instance ID	Name	State	Security groups
☒	i-0a1ede57ed707b746	Private_Server	Running	default
☒	i-0d627472082e88842	Public_Server	Running	default

Step 3: Create the Application Load Balancer

AWS Console:

- Go to **EC2 Dashboard** → **Load Balancers** → **Create Load Balancer** → **Application Load Balancer**.
- Basic Configuration:**
 - Load balancer name:** ASG-LB
 - Scheme:** internet-facing
 - IP address type:** IPv4

EC2 > Load balancers > Create Application Load Balancer

Create Application Load Balancer

The Application Load Balancer distributes incoming HTTP and HTTPS traffic across multiple targets such as Amazon EC2 instances, microservices, and containers, based on request attributes. When the load balancer receives a connection request, it evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.

► How Application Load Balancers work

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.

ASG-LB

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme | Info
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves internet-facing traffic.
- Has public IP addresses.
- DNS name resolves to public IPs.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name resolves to private IPs.
- Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type | Info
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**

Includes only IPv4 addresses.

☐ **Dualstack**

Includes IPv4 and IPv6 addresses.

☐ **Dualstack without public IPv4**

Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with **internet-facing** load balancers only.

3. Network Mapping:

- VPC:** ASG-VPC
- Mappings:** Select **both public subnets** (192.168.1.0/27 and 192.168.1.32/27).

Network mapping [Info](#)

The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC [Info](#)

The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#).

vpc-08505c7558333a4cd (ASG-VPC)
192.168.1.0/26



[Create VPC](#)

IP pools [Info](#)

You can optionally choose to configure an IPAM pool as the preferred source for your load balancers IP addresses. Create or view Pools in the [Amazon VPC IP Address Manager console](#).

☐ Use IPAM pool for public IPv4 addresses

The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.

Availability Zones and subnets [Info](#)

Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ us-east-1a (use1-az2)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.

subnet-080fbacd51e75d77a
IPv4 subnet CIDR: 192.168.1.0/27

Public_Subnet

☒ us-east-1b (use1-az4)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.

subnet-06ba86ce1d0633d7d
IPv4 subnet CIDR: 192.168.1.32/27

Private_Subnet



The selected subnet does not have a route to an internet gateway. This means that your load balancer will not receive internet traffic. You can proceed with this selection; however, for internet traffic to reach your load balancer, you must update the subnet's route table in the [VPC console](#).

4. Security Groups:

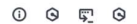
- Select ALB-SG.

5. Listeners and Routing:

- **Protocol:** HTTP
- **Port:** 80
- **Default action:** Forward to Apache-Target-Group.

6. Click Create Load Balancer.

[EC2](#) > [Load balancers](#) > Create Application Load Balancer



Listeners and routing [Info](#)

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

▼ Listener HTTP:80

[Remove](#)

Protocol

HTTP

Port

80

1-65535

Default action [Info](#)

The default action is used if no other rules apply. Choose the default action for traffic on this listener.

Routing action

☒ Forward to target groups

☐ Redirect to URL

☐ Return fixed response

Forward to target group [Info](#)

Choose a target group and specify routing weight or [create target group](#).

Target group

ASG-TG

Target type: Instance, IPv4 | Target stickiness: Off

HTTP



Weight

1

0-999

Percent

100%

+ Add target group

You can add up to 4 more target groups.

Target group stickiness [Info](#)

Enables the load balancer to bind a user's session to a specific target group. To use stickiness the client must support cookies. If you want to bind a user's session to a specific target, turn on the Target Group attribute Stickiness.

☐ Turn on target group stickiness

Listener tags - optional

Consider adding tags to your listener. Tags enable you to categorize your AWS resources so you can more easily manage them.

[Add listener tag](#)

You can add up to 50 more tags.

Step 4: Test the ALB

1. Get the ALB DNS name:

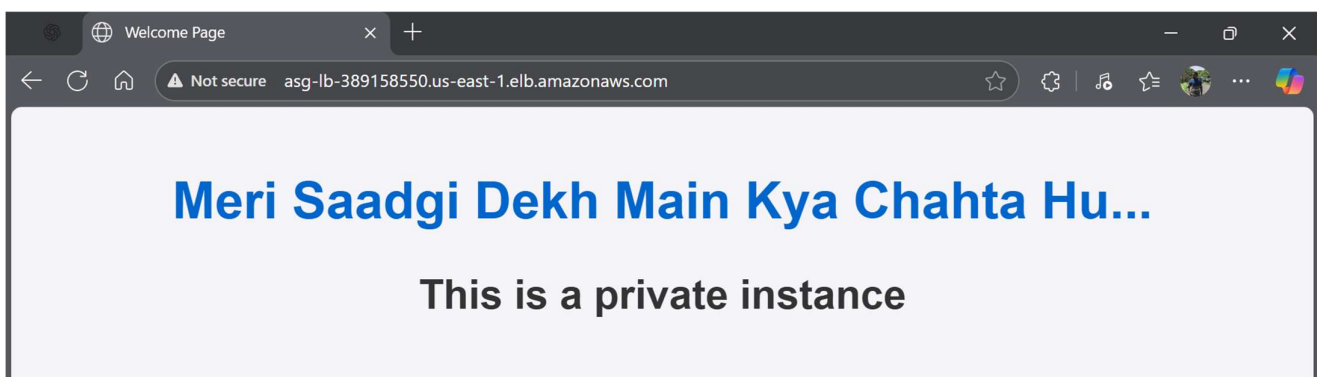
- Navigate to **EC2 > Load Balancers > Apache-ALB**.
- Copy the **DNS name** (e.g., Apache-ALB-123456789.us-east-1.elb.amazonaws.com).

2. Access the ALB:

- Open a web browser and navigate to:
`http://<ALB-DNS-Name>`
- Refresh the page to see traffic routed to **both public and private instances**.

3. Verify Target Health:

- Navigate to **Target Groups > Apache-Target-Group > Targets**.
- Both instances should show **Healthy**.



Target group: ASG-TG

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-08505c7558333a4cd
IP address type IPv4	Load balancer ASG-LB		
2 Total targets	2 Healthy 0 Anomalous	0 Unhealthy	0 Unused
		0 Initial	0 Draining

9. Store Application load balancer logs to S3.

Objective:

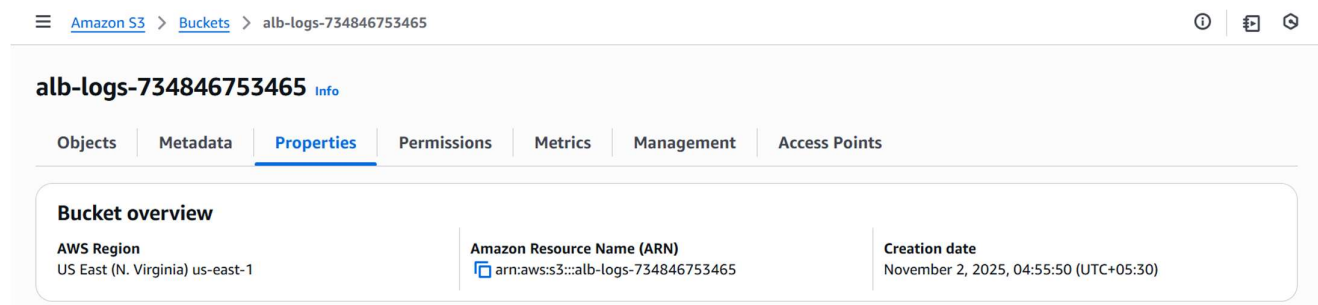
- Enable **ALB access logs** and store them in an **S3 bucket** for auditing and analysis.

Steps to store ALB logs in an S3 Bucket:

Step 1: Create an S3 Bucket for ALB Logs

1. AWS Console:

- Navigate to **S3 > Create Bucket**.
- **Bucket name:** <bucket_name> (must be globally unique).
- **Region:** US East (N. Virginia) us-east-1.
- **Block Public Access:** ☒ (Keep all settings enabled).
- Click **Create Bucket**.



Step 2: Attach a Policy to write ALB logs in the S3 Bucket:

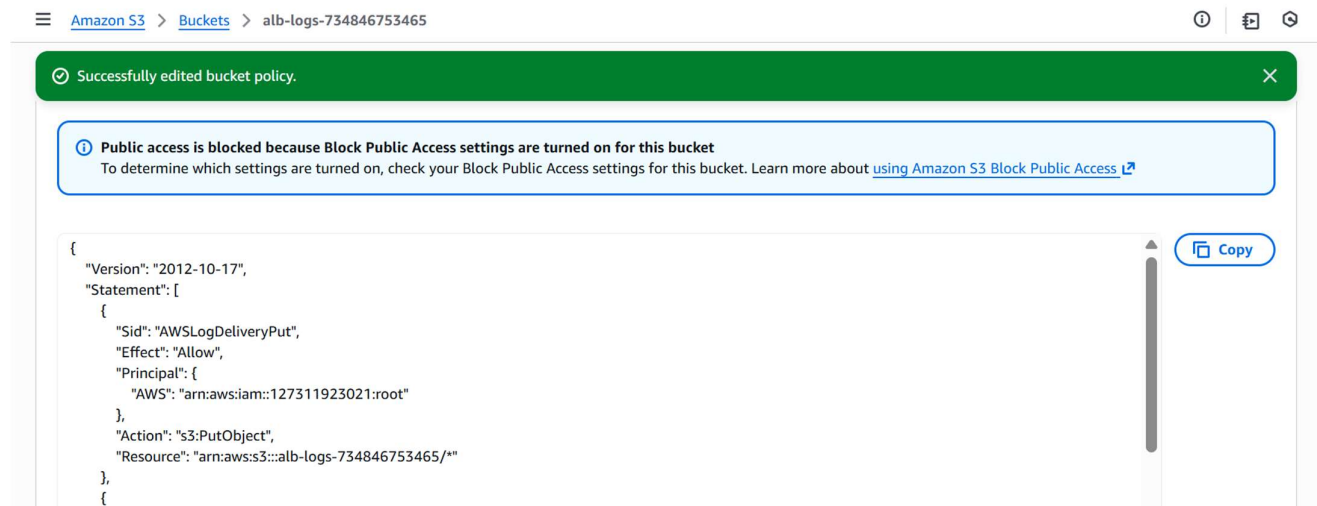
- Navigate to **S3 > alb-logs-734846753465 > Permissions > Bucket Policy**.
- Paste the below policy and save.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AWSLogDeliveryPut",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::127311923021:root"
      },
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::alb-logs-734846753465/*"
    },
    {
      "Sid": "AWSLogDeliveryPutACL",
      "Effect": "Allow",
      "Principal": {
```

```

    "Service": "delivery.logs.amazonaws.com"
  },
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::alb-logs-734846753465/*",
  "Condition": {
    "StringEquals": {
      "s3:x-amz-acl": "bucket-owner-full-control"
    }
  }
},
{
  "Sid": "AWSLogDeliveryGetACL",
  "Effect": "Allow",
  "Principal": {
    "Service": "delivery.logs.amazonaws.com"
  },
  "Action": "s3:GetBucketAcl",
  "Resource": "arn:aws:s3:::alb-logs-734846753465"
}
]
}

```



Step 3: Enable ALB Access Logs

1. AWS Console:

- Navigate to **EC2 > Load Balancers > Apache-ALB**.
- Select **Attributes** tab → **Edit Attributes**.
- **Access Logs:** ☒ Enable.
- **S3 Bucket:** alb-logs-734846753465.
- Click **Save Changes**.

Protection

☐ Deletion protection

To prevent your load balancer from being deleted accidentally, turn on deletion protection. If you turn on deletion protection, you must turn it off before you can delete the load balancer.

Monitoring

☒ Access logs

Access logs deliver detailed logs of all requests made to your Elastic Load Balancer. Choose an existing S3 location. If you don't specify a prefix, the logs are stored in the root of the bucket. Additional charges apply. [Learn more](#)

S3 URI

Q s3://alb-logs-734846753465



View

Browse S3

Format: s3://bucket/prefix

☐ Connection logs

Connection logs deliver detailed logs of all connections made to your Elastic Load Balancer. Choose an existing S3 location. If you don't specify a prefix, the logs are stored in the root of the bucket. Additional charges apply. [Learn more](#)

Cancel

Save changes

Step 4: Verify ALB Logs in S3

1. Navigate to **S3 > alb-logs-734846753465**.
2. After 5-10 minutes, you should see log files in the format:

```
734846753465_elasticloadbalancing_us-east-1_app.ASG-LB.f7b0cc5c8e762208_20251101T2355Z_34.225.58.130_645m5oh0.log.gz
http 2025-11-01T23:54:52.667060Z app/ASG-LB/f7b0cc5c8e762208 94.74.182.74:56462 192.168.1.25:80 0.001 0.001 0.000 200 200 183 974 "GET
http://34.225.58.130:80/ HTTP/1.1" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/42.0.2311.135 Safari/537.36
Edge/12.246" - - arn:aws:elasticloadbalancing:us-east-1:734846753465:targetgroup/ASG-TG/3fdac4765c24c9b3 "Root=1-69069dcc-7a455ebe049807a320086e09" "-" "-" 0
2025-11-01T23:54:52.665000Z "forward" "-" "-" "192.168.1.25:80" "200" "-" "-" TID_58548df6d0304649aea8e19a146b2936 "-" "-" "-"
```

734846753465_elasticloadbalancing_us-east-1_app.ASG-LB.f7b0cc5c8e762208_20251101T2355Z_34.225.58.130_645m5oh0.log.gz [Info](#)

Copy S3 URI

Download

Open

Object actions

Properties

Permissions

Versions

Object overview

Owner

iamomerfarooq257

AWS Region

US East (N. Virginia) us-east-1

Last modified

November 2, 2025, 05:25:08 (UTC+05:30)

Size

409.0 B

Type

gz

S3 URI

s3://alb-logs-734846753465/AWSLogs/734846753465/elasticloadbalancing/us-east-1/2025/11/01/734846753465_elasticloadbalancing_us-east-1_app.ASG-LB.f7b0cc5c8e762208_20251101T2355Z_34.225.58.130_645m5oh0.log.gz

Amazon Resource Name (ARN)

arn:aws:s3:::alb-logs-734846753465/AWSLogs/734846753465/elasticloadbalancing/us-east-1/2025/11/01/734846753465_elasticloadbalancing_us-east-1_app.ASG-LB.f7b0cc5c8e762208_20251101T2355Z_34.225.58.130_645m5oh0.log.gz

Entity tag (Etag)

d27f204411ee0a088f8f140a2ef101cd

10.Store the VPC flow logs in CloudWatch group.

Objective:

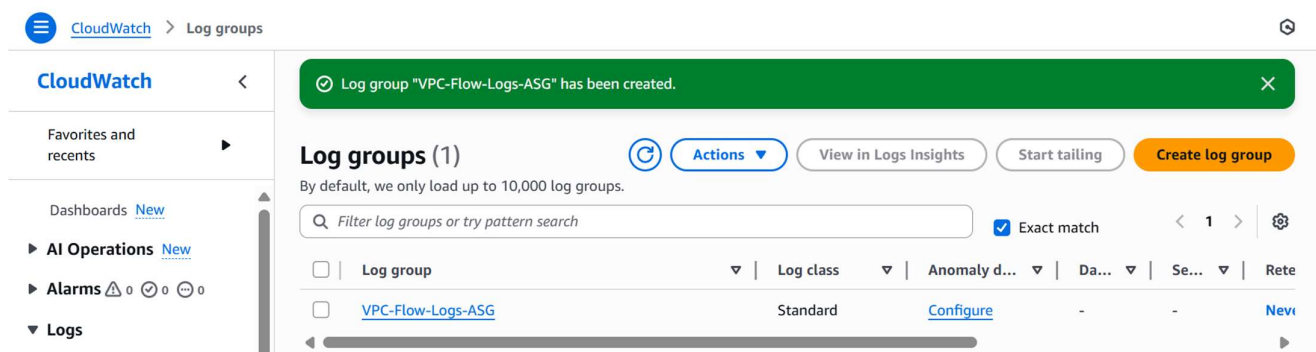
- Enable VPC Flow Logs to capture network traffic and store logs in CloudWatch Logs for monitoring and troubleshooting.

Steps to store VPC flow logs in CloudWatch group:

Step 1: Create a CloudWatch Logs Group

1. AWS Console:

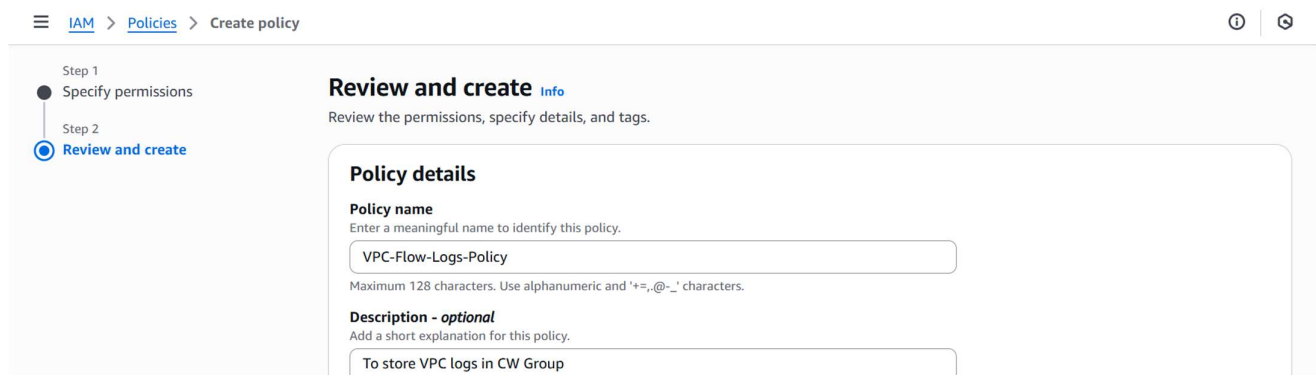
- Navigate to **CloudWatch > Logs > Log Groups > Create Log Group**.
- **Log group name:** VPC-Flow-Logs-ASG
- Click **Create**.



Step 2: Create an IAM Role for VPC Flow Logs

1. AWS Console:

- Navigate to **IAM > Roles > Create Role**.
- **Trusted entity type:** AWS Service.
- **Use case:** VPC Flow Logs.
- Click **Next**.
- **Permissions:** Attach the “**AmazonVPCFlowLogsCloudWatchLogs**” policy.



- **Role name:** VPC-Flow-Logs-Role.
- Click **Create Role**.

IAM > Roles > Create role

Step 1: Select trusted entity
Step 2: **Add permissions**
Step 3: Name, review, and create

Add permissions Info

Permissions policies (1/1083) Info

Choose one or more policies to attach to your new role.

Filter by Type: Customer managed 6 matches

Search

☐	Policy name	Type	Description
☐	Cloudtrail-CW-access-policy-Activity-Logs-b78cce76-b...	Customer man...	-
☐	Cloudtrail-CW-access-policy-Activity-Logs-c92ea781-3...	Customer man...	-
☐	Cloudtrail-CW-access-policy-CloudWatch-Logs-5c0511...	Customer man...	-
☐	Cloudtrail-CW-access-policy-test-logs-5314ab18-ce54-...	Customer man...	Policy for config CloudWathLogs for tr...
☐	CloudTrailCloudWatchLogsPolicy	Customer man...	-
☒	VPC-Flow-Logs-Policy	Customer man...	To store VPC logs in CW Group

Step 3: Enable VPC Flow Logs

1. AWS Console:

- Navigate to **VPC > Your VPCs > MyDevOps-VPC**.
- Select **Actions > Create Flow Log**.
- **Filter:** All.
- **Destination:** Send to CloudWatch Logs.
- **IAM Role:** Select VPC-Flow-Logs-Role.
- **Log Group:** /aws/vpc-flow-logs/MyDevOps-VPC.
- Click **Create Flow Log**.

VPC > Your VPCs > Create flow logs

Flow log settings

Name - optional

flow-logs-to-cw/group

Filter
The type of traffic to capture (accepted traffic only, rejected traffic only, or all traffic).
☐ Accept
☐ Reject
☒ All

Maximum aggregation interval Info
The maximum interval of time during which a flow of packets is captured and aggregated into a flow log record.
☒ 10 minutes
☐ 1 minute

Destination
The destination to which to publish the flow log data.
☒ Send to CloudWatch Logs
☐ Send to an Amazon S3 bucket
☐ Send to Amazon Data Firehose in the same account
☐ Send to Amazon Data Firehose in a different account

Destination log group Info
The name of an existing log group or the name of a new log group that will be created when you create this flow log. A new log stream is created for each monitored network interface.

[View this log group in the CloudWatch console](#)

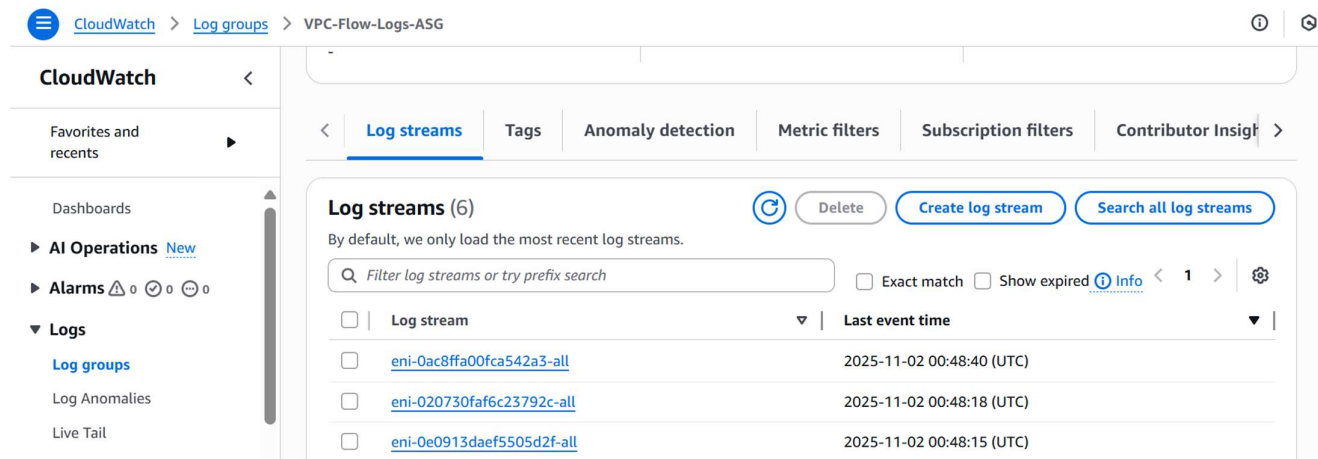
Service access
VPC flow logs require permissions to create log groups and publish events in CloudWatch.
☒ Use an existing service role
☐ Create and use a new service role

Service role Info
The IAM role that has permission to publish to the Amazon CloudWatch log group.

[View this service role in the IAM console](#)

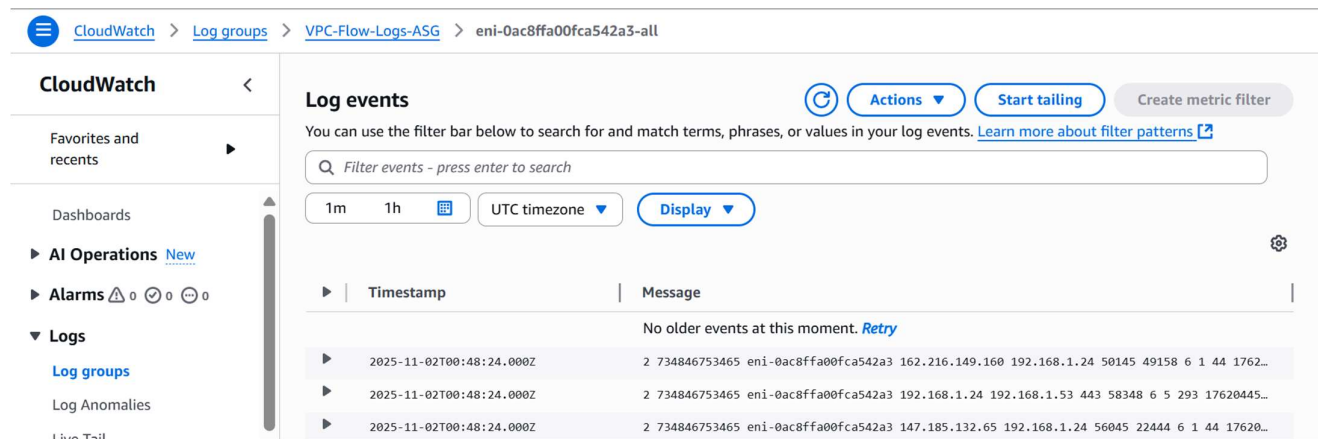
Step 4: Verify VPC Flow Logs in CloudWatch

1. Navigate to **CloudWatch > Logs > Log Groups > /aws/vpc-flow-logs/MyDevOps-VPC**.
2. Click on the latest log stream to view VPC traffic logs.



The screenshot shows the CloudWatch console interface. The breadcrumb navigation at the top reads: CloudWatch > Log groups > VPC-Flow-Logs-ASG. The left-hand navigation pane shows the 'Logs' section expanded, with 'Log groups' selected. The main content area is titled 'Log streams (6)' and includes a search bar, checkboxes for 'Exact match' and 'Show expired', and buttons for 'Delete', 'Create log stream', and 'Search all log streams'. Below this is a table of log streams.

☐	Log stream	Last event time
☐	eni-0ac8ffa00fca542a3-all	2025-11-02 00:48:40 (UTC)
☐	eni-020730faf6c23792c-all	2025-11-02 00:48:18 (UTC)
☐	eni-0e0913daef5505d2f-all	2025-11-02 00:48:15 (UTC)



The screenshot shows the CloudWatch console interface for a specific log stream. The breadcrumb navigation at the top reads: CloudWatch > Log groups > VPC-Flow-Logs-ASG > eni-0ac8ffa00fca542a3-all. The left-hand navigation pane is the same as in the previous screenshot. The main content area is titled 'Log events' and includes a search bar, a filter bar with '1m', '1h', and 'UTC timezone' options, and buttons for 'Actions', 'Start tailing', and 'Create metric filter'. Below this is a table of log events.

☐	Timestamp	Message
		No older events at this moment. Retry
☐	2025-11-02T00:48:24.000Z	2 734846753465 eni-0ac8ffa00fca542a3 162.216.149.160 192.168.1.24 50145 49158 6 1 44 1762...
☐	2025-11-02T00:48:24.000Z	2 734846753465 eni-0ac8ffa00fca542a3 192.168.1.24 192.168.1.53 443 58348 6 5 293 17620445...
☐	2025-11-02T00:48:24.000Z	2 734846753465 eni-0ac8ffa00fca542a3 147.185.132.65 192.168.1.24 56045 22444 6 1 44 17620...

11. Create Monitoring Dashboards to monitor CPU utilization and to monitor the Apache service.

Objective:

Create **CloudWatch Dashboards** to monitor:

- **EC2 CPU utilization.**
- **Apache service status** (via custom metrics).

Steps to create a monitoring dashboard to monitor CPU Utilization of the Public and Private Instances, and to monitor the Apache HTTPD service on both the servers:

Step 1: Install the CloudWatch Agent on EC2 Instances

1. **SSH into both EC2 instances** (public and private).
2. **Install the CloudWatch Agent:**

```
# yum install amazon-cloudwatch-agent -y
```

```
Installed:
  amazon-cloudwatch-agent-1.300059.1-1.amzn2023.x86_64
Complete!
[root@ip-192-168-1-25 ec2-user]#
```

```
Installed:
  amazon-cloudwatch-agent-1.300059.1-1.amzn2023.x86_64
Complete!
[root@ip-192-168-1-53 ec2-user]#
```

3. Configure the Agent:

- Create a configuration file:

```
# vi /opt/aws/amazon-cloudwatch-agent/bin/config.json -s
```

- Sample config.json (for CPU monitoring):

```
{
  "metrics": {
    "metrics_collected": {
      "cpu": {
        "measurement": [
          "cpu_usage_idle",
          "cpu_usage_iowait",
          "cpu_usage_user",
          "cpu_usage_system"
        ],
        "metrics_collection_interval": 60,
        "total_cpu": false
      }
    }
  }
}
```

4. Start the Agent:

```
# sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a start
```

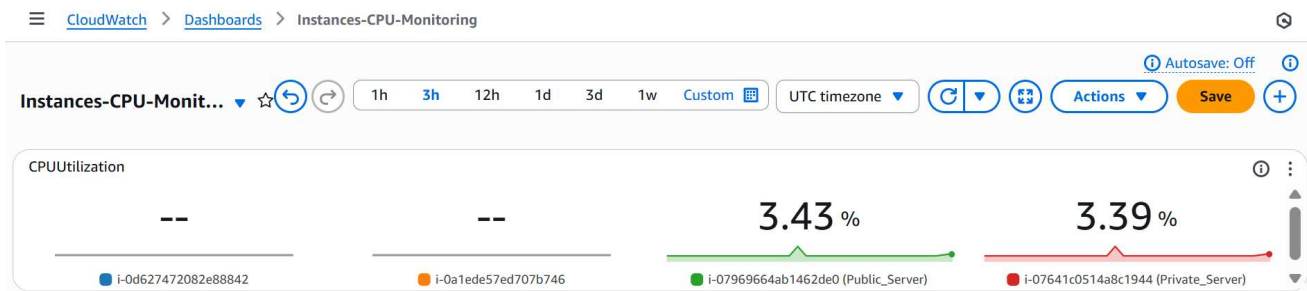
```
***** processing amazon-cloudwatch-agent *****  
amazon-cloudwatch-agent has already been started  
[root@ip-192-168-1-25 bin]#
```

```
***** processing amazon-cloudwatch-agent *****  
amazon-cloudwatch-agent has already been started  
[root@ip-192-168-1-53 bin]#
```

Step 2: Create a CloudWatch Dashboard for CPU Utilization

1. AWS Console:

- Navigate to **CloudWatch > Dashboards > Create Dashboard**.
- **Dashboard name:** EC2-CPU-Monitoring.
- Click **Add Widget > Metrics > EC2 > Per-Instance Metrics**.
- Select **CPUUtilization** for both instances.
- Click **Create Widget**.
- Save the dashboard.



Step 3: Monitor Apache Service Status (Custom Metrics)

1. Create a Custom Script to Check Apache Status:

- SSH into both instances and create a script:

```
# vi /usr/local/bin/check_apache.sh
```
- Paste the following:

```
#!/bin/bash  
STATUS=$(systemctl is-active httpd)  
if [ "$STATUS" = "active" ]; then  
    echo "Apache is running"  
    aws cloudwatch put-metric-data --metric-name ApacheStatus --namespace Custom --value 1 --dimensions  
    InstanceId=$(curl -s http://169.254.169.254/latest/meta-data/instance-id) --region us-east-1  
else  
    echo "Apache is NOT running"
```

```
aws cloudwatch put-metric-data --metric-name ApacheStatus --namespace Custom --value 0 --dimensions
InstanceId=$(curl -s http://169.254.169.254/latest/meta-data/instance-id) --region us-east-1
fi
```

- Make the script executable:

```
# chmod +x /usr/local/bin/check_apache.sh
```

2. Add a Cron Job to Run the Script Every 5 Minutes:

```
# crontab -e
```

- Add the following line:

```
*/5 * * * * /usr/local/bin/check_apache.sh
```

```
Installed:
cronie-1.5.7-1.amzn2023.0.2.x86_64
cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64

Complete!
[root@ip-192-168-1-25 bin]# crontab -e
bash: crontab: command not found
[root@ip-192-168-1-25 bin]# crontab -e
no crontab for root - using an empty one
crontab: installing new crontab
[root@ip-192-168-1-25 bin]# |
```

```
=====
Installed:
cronie-1.5.7-1.amzn2023.0.2.x86_64
cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64

Complete!
[root@ip-192-168-1-53 bin]# crontab -e
no crontab for root - using an empty one
crontab: installing new crontab
[root@ip-192-168-1-53 bin]#
```

3. Create a CloudWatch Dashboard for Apache Status:

- Navigate to **CloudWatch > Dashboards > Create Dashboard**.
- **Dashboard name:** Apache-Service-Monitoring.
- **Add Widget > Custom Metrics > Custom > ApacheStatus**.
- Save the dashboard.

12.CPU utilizations more than 70% then it should trigger Autoscaling and launch a new instance.

Objective:

Create an Auto Scaling Group (ASG) that:

- Launches a new EC2 instance when **CPU utilization > 70%** for 5 minutes.
- Uses the **same Apache setup** as existing instances.

Steps to create an ASG and launch an Instance upon exceeding CPU Utilization:

Step 1: Create a Launch Template

1. AWS Console:

- Navigate to **EC2 > Launch Templates > Create Launch Template**.
- **Name:** Apache-Launch-Template.
- **AMI:** Amazon Linux 2 (ami-0c55b159cbfafa1f0).
- **Instance Type:** t2.micro.
- **Key Pair:** Select your existing key pair.
- **Security Groups:** Select the **security group** used for your existing instances.
- **User Data:** Paste the following to auto-install Apache:

```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
echo "<h1>Auto-Scaled Apache Instance</h1>" > /var/www/html/index.html
```

[illegible]

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.

Launch template name - *required*

Apache-Server-Template

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '@'.

ASG Public Server (Apache HTTPD)

Max 255 chars

Select this if you intend to use this template with EC2 Auto Scaling

☐ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

► [Source template](#)

```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
echo "<h1>Auto-Scaled Apache Instance</h1>" > /var/www/html/index.html
```

☐ User data has already been base64 encoded

Software Image (AMI)

Amazon Linux 2023 kernel-6.1 A...[read more](#)
ami-0bdd88bd06d16ba03

Virtual server type (instance type)

t2.micro

Firewall (security group)

SG-Instances

Storage (volumes)

1 volume(s) - 8 GiB

- Navigate to **EC2 > Auto Scaling > Auto Scaling Groups > Create Auto Scaling Group**.
- **Name:** Apache-ASG.
- **Launch Template:** Select Apache-Launch-Template.

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 1: Choose launch template

Choose launch template [Info](#)

Specify a launch template that contains settings common to all EC2 instances that are launched by this Auto Scaling group.

Name

Auto Scaling group name
Enter a name to identify the group.
Apache-ASG
Must be unique to this account in the current Region and no more than 255 characters.

Launch template [Info](#)

For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.

Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

Apache-Server-Template

[Create a launch template](#)

Version
Default (1)

[Create a launch template version](#)

Description ASG Public Server (Apache HTTPD)	Launch template Apache-Server-Template lt-0e351e8935535f185	Instance type t2.micro
--	--	----------------------------------

- **VPC:** MyDevOps-VPC.
- **Subnets:** Select **both private subnets** (10.0.2.0/24 and 10.0.4.0/24).

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 7: Review

Network [Info](#)

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.
vpc-08505c7558333a4cd (ASG-VPC)
192.168.1.0/26

[Create a VPC](#)

Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

use1-az2 (us-east-1a) subnet-080fbacd51e75d77a (Public_Subnet) 192.168.1.0/27	X
use1-az4 (us-east-1b) subnet-06ba86ce1d0633d7d (Private_Subnet) 192.168.1.32/27	X

[Create a subnet](#)

Availability Zone distribution - new
Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

☒ **Balanced best effort**
If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

☐ **Balanced only**
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

- **Load Balancing:** Attach to Apache-ALB.
- **Health Checks:** Use ELB health checks.

- Step 1
● Choose launch template
- Step 2
● Choose instance launch options
- Step 3 - optional
● **Integrate with other services**
- Step 4 - optional
● Configure group size and scaling
- Step 5 - optional
● Add notifications
- Step 6 - optional
● Add tags
- Step 7
● Review

Integrate with other services - optional [Info](#)

Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.

Load balancing [Info](#)

Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

Select Load balancing options

- ☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.
- ☒ **Attach to an existing load balancer**
Choose from your existing load balancers.
- ☐ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to an existing load balancer

Select the load balancers to attach

- ☒ **Choose from your load balancer target groups**
This option allows you to attach Application, Network, or Gateway Load Balancers.
- ☐ Choose from Classic Load Balancers

Existing load balancer target groups

Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups

ASG-TG | HTTP
Application Load Balancer: ASG-LB

- **Group Size:** Desired: 1, Minimum: 1, Maximum: 3.
- Click **Next**.

- Step 1
● Choose launch template
- Step 2
● Choose instance launch options
- Step 3 - optional
● Integrate with other services
- Step 4 - optional
● **Configure group size and scaling**
- Step 5 - optional
● Add notifications
- Step 6 - optional
● Add tags
- Step 7
● Review

Configure group size and scaling - optional [Info](#)

Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.

Group size [Info](#)

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type

Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity

Specify your group size.

1

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

1

Equal or less than desired capacity

Max desired capacity

3

Equal or greater than desired capacity

2. Configure Scaling Policies:

- **Scaling Policy Name:** CPU-Utilization-Alarm.
- **Metric Type:** Average CPU Utilization.

- **Target Value:** 70%.
- **Instance Warm-up:** 300 seconds.
- **Add Policy.**

EC2 > Auto Scaling groups > Create Auto Scaling group

Automatic scaling - optional

Choose whether to use a target tracking policy | Info

You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☐ **No scaling policies**
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☒ **Target tracking scaling policy**
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Scaling policy name

CPU-Utilization-Alarm

Metric type | Info

Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.

Average CPU utilization

Target value

70%

Instance warmup | Info

300 seconds

☐ Disable scale in to create only a scale-out policy

- **Create an SNS notification**

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 1 Choose launch template

Step 2 Choose instance launch options

Step 3 - optional Integrate with other services

Step 4 - optional Configure group size and scaling

Step 5 - optional **Add notifications**

Step 6 - optional Add tags

Step 7 Review

Add notifications - optional | Info

Send notifications to SNS topics whenever Amazon EC2 Auto Scaling launches or terminates the EC2 instances in your Auto Scaling group.

▼ Notification 1 Remove

Send a notification to

CPU-Utilization-Alert

With these recipients

iamshaikomerfarooq@gmail.com

Use existing topic

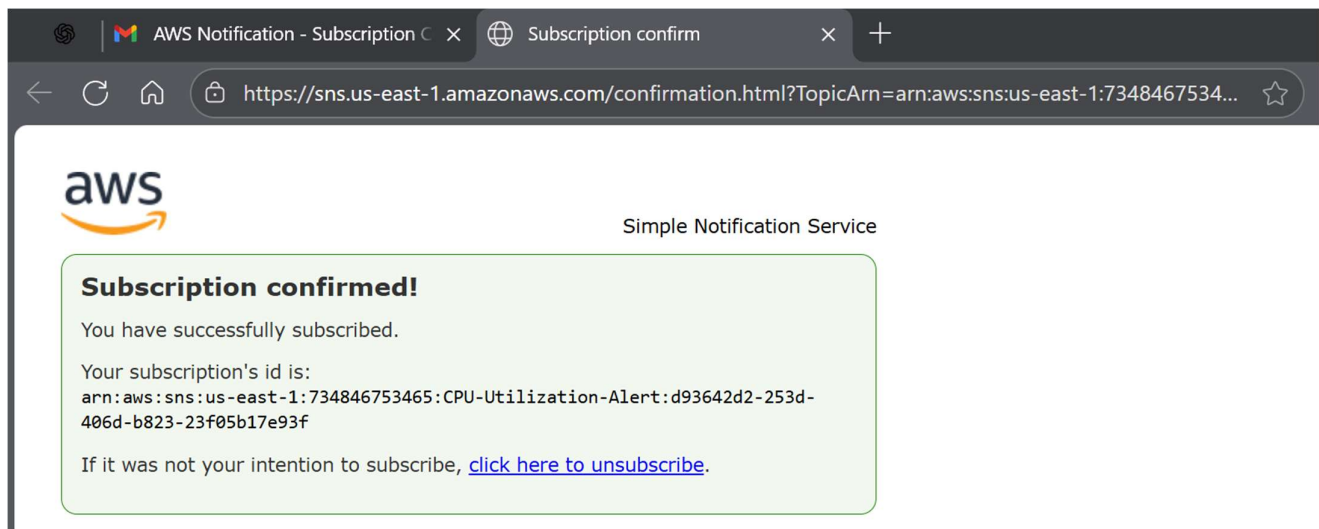
Event types

Notify subscribers whenever instances

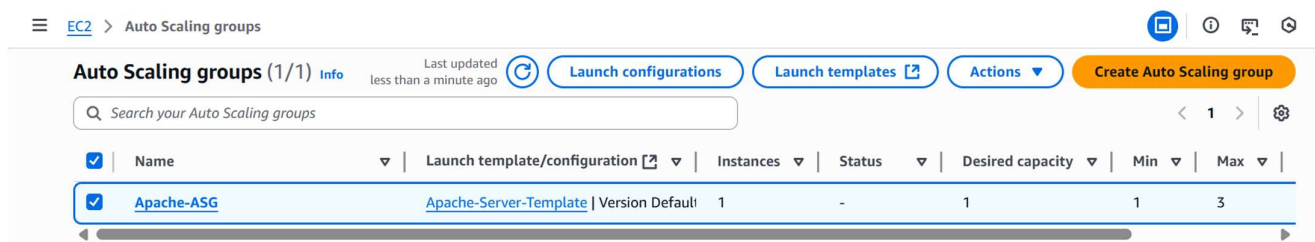
☒ Launch
☒ Terminate
☒ Fail to launch
☒ Fail to terminate

Add notification

Cancel Skip to review Previous Next



3. Review and Create ASG.



Step 3: Test Auto Scaling

1. Simulate High CPU Load on an Instance:

- SSH into one of the instances and run:
`# stress --cpu 2 --timeout 600`
- This will spike CPU usage to **100%** for 10 minutes.

The collage consists of four terminal windows from an Amazon Linux 2023 EC2 instance:

- Top Left:** Shows the Amazon Linux logo and the URL <https://aws.amazon.com/linux/amazon-linux-2023>. It also displays the last login time: "Sun Nov 2 04:08:39 2025 from 192.168.1.25".
- Top Right:** Displays the output of the `top` command, showing system uptime (4 days, 12:44), user count (22), and resource usage: 193 tasks total (5 running, 188 sleeping), 98.0% CPU, 949.4 MiB memory, and 0.0 MiB swap.
- Bottom Left:** Shows the output of the `stress --cpu 4 --time out 600s` command, indicating it is dispatching hogs: 4 CPU, 0 IO, 0 VM, and 0 HDD.
- Bottom Right:** Displays the output of the `top` command again, showing updated resource usage: 156 tasks total (5 running, 151 sleeping), 97.7% CPU, 949.4 MiB memory, and 0.0 MiB swap.

- Check the **Target Group** in the ALB to confirm the new instance is registered and healthy.